

Lustjinn

Python as a backend, for a .NET developer

A course that grows with the code. This edition was built on 1 October 2026.

Contents

0 Orientation	3
0.1 How to read this course	3
0.2 The map	3
0.3 Break 1 — Types are checked, but not enforced	4
0.4 Break 2 — There is no build step	5
0.5 Break 3 — Every project gets its own interpreter environment	5
0.6 Break 4 — Async runs on one thread	6
0.7 Smaller differences you will meet soon	6
0.8 What is already on your machine	7
0.9 What is next	7

CHAPTER 0

Orientation

You already know how to build a backend. You know what a request pipeline is, what dependency injection buys you, why migrations exist, how `async/await` frees a thread. None of that changes. What changes is the vocabulary, the tooling, and a handful of assumptions so deep in .NET that you no longer notice them — and those are the ones that bite.

This course follows **Lustjinn** as it is built: a web app for roleplay with a memory that does not forget, the Python successor to `airp`. Each chapter is one step of the roadmap. Claude writes the code; the chapter explains it — what each new piece is, why it is there, and what it would be in .NET. When a chapter and the code disagree, the chapter is out of date: that is a bug.

0.1 How to read this course

Every chapter has the same shape: what we are building in this step, the code that was written, the concepts it introduced, and what changed in the repository. Four kinds of box recur:

THE .NET LENS

Maps a Python idea onto the .NET one you already know — and says where the analogy breaks.

WORTH KNOWING

Background that is useful but not needed to follow the step.

WATCH OUT

A trap. Usually one that would compile — or rather, *run* — without complaint.

TRY IT

A small exercise. Optional, but the fastest way to make a concept stick.

0.2 The map

Most of the Python toolchain has a direct counterpart in what you use every day. Keep this table nearby; later chapters assume it.

You know (.NET)	Lustjinn uses (Python)	What it is
<code>dotnet CLI</code>	uv	Creates projects, installs packages, runs things
<code>nuget.org</code>	PyPI	The public package registry
<code>.csproj</code>	<code>pyproject.toml</code>	The project file: name, dependencies, tool settings
<code>packages.lock.json</code>	<code>uv.lock</code>	Exact resolved versions, committed
<code>global.json</code>	<code>.python-version</code>	Pins the runtime version
Roslyn analyzers, <code>dotnet format</code>	ruff	Linters and formatters in one
The compiler + nullable reference types	pyright (strict)	The type checker
xUnit	pytest	Tests
NSubstitute, a fake <code>HttpMessageHandler</code>	respx	Fakes HTTP calls in tests
Kestrel	uvicorn	The web server that runs the app
ASP.NET Core minimal APIs	FastAPI	Routing, model binding, DI, OpenAPI
<code>record</code> + <code>DataAnnotations</code>	Pydantic models	Typed data with validation
<code>IOptions<T></code>	pydantic-settings	Typed configuration from the environment
EF Core	SQLAlchemy 2.0	ORM and query builder
EF Core migrations	Alembic	Schema migrations
<code>HttpClient</code>	httpx	HTTP client, sync and async
<code>Task</code> , <code>async / await</code>	asyncio coroutines	Asynchronous code

The table is a starting point, not a promise. The rest of this chapter covers the four places where the mapping breaks hardest.

0.3 Break 1 — Types are checked, but not enforced

In C#, a type is a contract the compiler enforces and the runtime relies on. In Python, a type hint is an *annotation*: the interpreter stores it and otherwise ignores it.

```
def greet(name: str) -> str:
    return "Hello, " + name

greet(42)  # Runs. Fails only inside, at the "+", with a TypeError.
```

Nothing stops that call at run time. What stops it is **pyright**, run before the code ships — exactly the role the C# compiler plays, except that it is a separate tool and you could skip it. Lustjinn runs pyright in **strict** mode, in CI, on every push: that is our `TreatWarningsAsErrors`.

THE .NET LENS

`str | None` is Python's `string?`. With nullable reference types on, C# warns when you dereference something that might be null; pyright strict does the same for `None`. The difference is that C#'s `null` checks are warnings you opted into, while in strict pyright they are errors from the start.

WORTH KNOWING

Some libraries *do* read type hints at run time, on purpose. **Pydantic** and **FastAPI** are the two you will meet first: a parameter declared `limit: int` makes FastAPI parse and validate the query string as an integer and return a 422 if it is not. That is model binding, driven by the same annotations pyright checks.

0.4 Break 2 — There is no build step

`dotnet build` turns your project into an assembly; `dotnet run` runs the assembly. Python has no equivalent step you run yourself. The interpreter (**CPython**, the standard one) compiles each file to bytecode the first time it is imported, caches it in a `__pycache__` folder, and executes it. A syntax error surfaces when the file is loaded; a typo in a name surfaces only when that line runs.

That is why the tooling matters more than in .NET: **ruff** and **pyright** are the build. If they are clean and the tests pass, the code is as checked as it will get before it runs.

0.5 Break 3 — Every project gets its own interpreter environment

NuGet keeps packages in a global cache, and each project references the versions it wants; two projects can use different versions of the same package without noticing each other.

Python installs packages *into an interpreter*. Install a package into the system Python and every project on the machine sees that version. The answer is a **virtual environment**: a folder (`.venv`) holding a private copy of the interpreter's package directory, one per project.

uv manages all of it. It downloads the Python version the project pins, creates `.venv`, installs exactly what `uv.lock` says, and runs commands inside that environment:

<code>uv sync</code>	<code># make .venv match uv.lock</code>	<code>≈ dotnet restore</code>
<code>uv add httpx</code>	<code># add a dependency, update the lock</code>	<code>≈ dotnet add package</code>
<code>uv run pytest</code>	<code># run a command inside .venv</code>	<code>≈ dotnet test</code>
<code>uv python install</code>	<code># fetch the pinned Python version</code>	<code>≈ installing an SDK</code>

WATCH OUT

Your Windows machine has an older system Python (3.9) on the `PATH`. Lustjinn never uses it: `uv` fetches the version `.python-version` asks for and keeps it separate. If a command ever behaves as if it were on 3.9, it was run with `python` instead of `uv run python`.

0.6 Break 4 — Async runs on one thread

In ASP.NET Core, each request runs on a thread-pool thread. If a handler blocks — a synchronous database call, a `Thread.Sleep` — it wastes that one thread, and the pool covers for it.

Python's `asyncio` runs every coroutine on **a single thread**, inside an **event loop**. An `await` hands control back to the loop so another request can make progress. A call that blocks without awaiting does not waste one thread; it freezes *every* request the server is handling until it returns.

```
async def handler():
    time.sleep(2)           # Blocks the event loop: the whole server stops for 2 s.
    await asyncio.sleep(2)  # Yields to the loop: other requests keep running.
```

This is why Lustjinn uses the async versions of everything that waits: **httpx**'s `AsyncClient`, SQLAlchemy's async engine, an async Postgres driver.

THE .NET LENS

`async def` $\approx$ an `async Task` method, and `await` means the same thing. The difference is what happens *between* awaits: in .NET, other work runs on other threads; in Python, other work runs only when you await. There is no `ConfigureAwait`, and no `SynchronizationContext` to think about.

WORTH KNOWING

FastAPI hides one sharp edge for you. An endpoint declared with plain `def` (not `async def`) is run on a separate thread pool, so blocking inside it is safe. An endpoint declared `async def` runs on the event loop and must never block.

0.7 Smaller differences you will meet soon

- **Files are modules; folders are packages.** There is no `namespace` keyword and no project reference: `from lustjinn.stories import Story` imports by path from the package folder.
- **Naming:** `snake_case` for functions, variables and modules; `PascalCase` for classes; `UPPER_CASE` for constants. The leading underscore (`_helper`) means "private by convention" — nothing enforces it.
- **self is explicit.** A method's first parameter is the instance, written out. The constructor is `__init__`.
- **Indentation is syntax.** Blocks have no braces; the indentation *is* the block. `ruff`'s formatter keeps it consistent.
- `None`, `True`, `False` are capitalised.

0.8 What is already on your machine

The setup for this course and for step 1 is installed:

- **uv** — the Python toolchain. It will fetch the current Python on first use.
- **pandoc** and **Typst** — they turn the Markdown chapters of this course into this PDF.

The course lives in the repository under `docs/course/`. To rebuild it:

```
./docs/course/build.ps1          # the dark edition
./docs/course/build.ps1 -Light   # a light edition, for printing
```

TRY IT

Run `uv --version` and `uv python list`. The second command lists the Python versions uv can install, and any it already has. Next chapter, one of them becomes Lustjinn's.

0.9 What is next

Chapter 1 — Hello, deployed. A uv project, FastAPI with one endpoint, one pytest test, ruff and pyright in CI, and the whole thing deployed to Render — so that `GET /health` answers from the internet. It teaches the project layout, uv in practice, and how a Python web app is actually served.